IoT Smart Home Controller System

Software Requirements Specification (SRS)

Submitted By:

Usman Rafique (191)
Muhammad Muneeb (013)
Husnain Raza (163)

Section:

D

Submitted To:

Ma’am Jaweria Ramzan

Table of Contents

1. Introduction

1.1 Purpose of the Document

This Software Requirements Specification (SRS) document formally defines the functional and non-functional requirements of the IoT Smart Home Controller System. It serves as the primary reference artifact for all subsequent phases of the Software Verification and Validation (SVV) semester project, including formal specification in Z Notation, VDM (Vienna Development Method), Alloy modeling, validation testing, and security verification. The document is intended for use by software engineers, project supervisors, formal verification analysts, and academic evaluators.

The SRS adheres to established software engineering standards and is structured to support a rigorous, methodology-driven development lifecycle. Every requirement, constraint, invariant, and operation defined herein has been carefully constructed to be formally verifiable and internally consistent.

1.2 Scope of the System

The IoT Smart Home Controller System is a formally specified, simulation-based smart home management system. The scope of this project is restricted to formal modeling, state-based specification, and verification and validation of a centralized smart home controller. The system does not involve physical hardware, real sensor integration, cloud services, artificial intelligence, or machine learning components.

The system automates and monitors the following four categories of smart home functionality:

- Smart Lights: Control of residential lighting via authenticated user commands.
- Smart Door Locks: Remote locking and unlocking of a door via secure user operations.
- Smart Alarms: Activation and deactivation of a security alarm, including automatic triggering upon intrusion detection.
- Temperature Monitoring: Continuous monitoring of ambient temperature with automated alerts when a safety threshold is exceeded.

1.3 Overview of the IoT Smart Home Controller System

The IoT Smart Home Controller System is a centralized, state-based control platform designed to manage discrete smart devices within a residential environment. The system exposes a set of formally defined operations to authenticated users and administrators. All device interactions are mediated through a centralized controller that maintains the current state of each device, enforces access control policies, evaluates system invariants, and generates alerts in response to predefined triggering events.

The system is composed of five primary functional modules: User Authentication, Light Control, Door Lock Management, Alarm Management, and Temperature Monitoring. Each module is independently specified with clearly defined states, transitions, and operations, making the system amenable to formal verification using Z Notation, VDM, and Alloy.

1.4 Importance of Formal Verification in Smart Systems

Smart home systems operate in safety-critical and security-sensitive environments where incorrect behavior can result in security breaches, property damage, or personal harm. Traditional testing methodologies, while necessary, are insufficient to guarantee correctness under all possible operational conditions. Formal verification provides mathematical proof of system correctness by rigorously checking that all defined properties, invariants, and behavioral contracts are satisfied under every reachable system state.

The application of formal methods such as Z Notation ensures that system states are precisely defined, that all operations are accompanied by unambiguous preconditions and postconditions, and that invariants are never violated. VDM further supports data refinement and operational specification, while Alloy enables automated constraint solving to detect specification flaws prior to implementation. Together, these techniques yield a provably correct system specification.

1.5 Objectives of the Project

The primary objectives of this project are outlined in detail in Section 2.

2. Project Objectives

The IoT Smart Home Controller System project pursues the following formally aligned objectives:

1. Automate Smart Home Device Control: Define and specify the complete set of state transitions and control operations for smart lights, door locks, and alarms, enabling deterministic and verifiable automated control.
2. Improve System Security: Specify and enforce access control mechanisms that prevent unauthorized users from interacting with any smart home device. Define security invariants and authentication preconditions for all operations.
3. Ensure System Correctness Through Formal Methods: Construct the system specification in a manner that is fully compatible with Z Notation schemas, VDM pre/postcondition specifications, and Alloy relational models. Guarantee that no valid sequence of operations leads to an invalid or inconsistent system state.
4. Prevent Inconsistent and Invalid States: Define explicit state invariants that prohibit logically contradictory device states (e.g., a door simultaneously LOCKED and UNLOCKED). Enforce these invariants at the specification level.
5. Apply Formal Verification Techniques: Verify system correctness by modeling operations and invariants in Z Notation and Alloy, and demonstrate that all reachable states satisfy the system's safety and security properties.
6. Validate Operational Behavior: Define and execute a structured validation checklist that confirms each system operation produces the expected output state given its preconditions.
7. Perform Security Analysis: Identify and address security requirements aligned with OWASP guidelines, and verify that the system specification adequately prevents common vulnerability classes including unauthorized access, input injection, and session mismanagement.

8. Produce a Comprehensive and Traceable SRS: Deliver a formal, professional Software Requirements Specification that establishes full traceability between requirements, states, operations, invariants, use cases, and validation criteria.

3. System Overview

3.1 Overall Architecture

The IoT Smart Home Controller System is architected as a centralized, event-driven, state-based system. A single Smart Home Controller acts as the central authority for all device management operations. Users and administrators interact with the controller through a defined interface layer. The controller evaluates incoming requests, validates user authentication status, checks system invariants, executes the requested state transitions, and returns appropriate responses.

The architecture is logically divided into three layers: the Interaction Layer (handling user input and output), the Control Layer (the Smart Home Controller responsible for enforcing rules and executing operations), and the Device State Layer (maintaining the canonical state of each smart device). This separation facilitates independent formal specification of each layer.

3.2 System Components

The system comprises the following components:

- Smart Home Controller: The central component responsible for processing all user requests, enforcing authentication, validating invariants, and executing device state transitions.
- User Authentication Component: Validates user credentials and manages session state.
- Device State Store: Maintains the current state of each smart device (Light, Door, Alarm, Temperature) as a formal state record.
- Alert Generator: Monitors system events and generates notifications for intrusion detection and temperature threshold violations.
- Audit Logger: Records all device commands, state changes, and security events for administrative review.

3.3 System Modules

3.3.1 User Authentication Module

This module manages the lifecycle of user sessions. It accepts a username and password, validates the credentials against the stored user record, and issues a session token upon successful authentication. The module enforces a maximum of five consecutive failed login attempts before imposing a temporary account lockout. All authentication events are logged.

3.3.2 Light Control Module

This module manages the state of the smart light. It accepts TurnOnLight() and TurnOffLight() commands from authenticated users. The module verifies that the requested transition is valid (i.e., the light is not already in the target state), executes the transition, and updates the device state store. The light state is always either ON or OFF.

3.3.3 Door Lock Module

This module manages the state of the smart door lock. It accepts LockDoor() and UnlockDoor() commands from authenticated users. The module validates that the door is in the expected pre-transition state, executes the transition, and updates the state store. The module enforces the invariant that the door is always in exactly one of LOCKED or UNLOCKED state. Unauthorized attempts to alter the door state trigger the intrusion detection mechanism.

3.3.4 Alarm Module

This module manages the smart alarm. It accepts ActivateAlarm() and DeactivateAlarm() commands from authenticated users or administrators. Additionally, it receives intrusion events from the Door Lock Module and autonomously activates the alarm without requiring a user command. The alarm state is always either ACTIVE or INACTIVE.

3.3.5 Temperature Monitoring Module

This module continuously reads from a simulated temperature sensor and evaluates the reading against a predefined safety threshold (40°C by default). If the reading exceeds the threshold, the module transitions the temperature state to HIGH and dispatches a temperature alert. If the reading returns to or below the threshold, the state reverts to SAFE. Invalid or null sensor readings are rejected and do not alter the system state.

4. System Actors

4.1 Actor: User

The User actor represents any authenticated residential occupant who interacts with the Smart Home Controller to manage smart devices. The User is granted access to device control operations upon successful authentication.

Responsibilities:

- Provide valid credentials during the login process.
- Issue device control commands (light, door, alarm) through the system interface.
- Receive notifications and alerts generated by the system.
- Log out of the system upon completion of interaction.

Permissions:

- Execute Login(), Logout(), TurnOnLight(), TurnOffLight(), LockDoor(), UnlockDoor(), ActivateAlarm(), DeactivateAlarm().
- View current device states and temperature readings.
- Receive alert notifications.

Restrictions:

- Cannot access administrative functions such as user management or full audit logs.
- Cannot issue device commands without an active authenticated session.

4.2 Actor: Admin

The Admin actor represents a privileged system operator with elevated access rights. The Admin is responsible for system oversight, user management, and emergency device control.

Responsibilities:

- Monitor overall system health and device status.
- Review audit logs for security events and device state changes.
- Manage user accounts within the system.
- Execute emergency device resets if the system enters an inconsistent state.

Permissions:

- All operations available to the User actor.
- Execute ResetDevice() operations on any device.
- View device state logs and authentication audit trails.
- Override device states in emergency scenarios.

Restrictions:

- Admin operations are also subject to authentication requirements.
- Admins may not violate system invariants; invalid override commands are rejected.

5. Functional Requirements

The following table defines all functional requirements of the IoT Smart Home Controller System. Each requirement is assigned a unique identifier, a complete description, the expected inputs and outputs, and a priority classification. These requirements form the foundation for formal Z Notation schemas, VDM operations, and Alloy assertions defined in subsequent verification phases.

Req. ID	Description	Input	Output	Priority
FR-01	The system shall authenticate users via a unique username and password combination before granting access to any smart home control functionality.	Username, Password	Authentication success or failure message; session token upon success	High
FR-02	The system shall allow an authenticated user to turn the smart light ON or OFF through a designated control interface.	User command (ON/OFF), Device ID	Updated light state (ON or OFF); confirmation message	High
FR-03	The system shall enable an authenticated user to lock or unlock the smart door lock remotely.	User command (LOCK/UNLOCK), Device ID	Updated door state (LOCKED or UNLOCKED); confirmation message	High
FR-04	The system shall permit an authenticated user or admin to activate or deactivate the smart alarm.	User command (ACTIVATE/DEACTIVATE)	Updated alarm state (ACTIVE or INACTIVE); confirmation message	High
FR-05	The system shall continuously monitor the temperature sensor and store the current temperature reading.	Temperature sensor value (numeric)	Current temperature reading; alert if threshold exceeded	High

FR-06	The system shall maintain and persistently store the current state of each smart device (light, door, alarm) at all times.	Device state change events	Updated device state stored in the system state record	High
FR-07	The system shall restrict any device control operations to authenticated users only; unauthorized access attempts shall be rejected.	Unauthenticated request	Access denied response; log entry of unauthorized attempt	Critical
FR-08	The system shall generate an intrusion alert and activate the alarm automatically when an unauthorized access attempt on the door lock is detected.	Unauthorized door command event	Alarm state changes to ACTIVE; intrusion alert generated	Critical
FR-09	The system shall generate a temperature alert and notify the user when the measured temperature exceeds the predefined safety threshold (e.g., 40°C).	Temperature value > threshold	Temperature alert notification; temperature state changes to HIGH	High
FR-10	The system shall provide the admin with the ability to view full device status	Admin credentials, log query	Device status log; user management interface	Medium

	logs and manage user accounts.			
FR-11	The system shall prevent a door from being simultaneously in LOCKED and UNLOCKED states.	Conflicting state command	Operation rejected; error message displayed	Critical
FR-12	The system shall allow the admin to reset any device to its default safe state in case of failure or inconsistency.	Admin reset command, Device ID	Device reverts to default state; confirmation message	Medium

6. Non-Functional Requirements

Non-functional requirements define the qualitative properties and operational constraints that the IoT Smart Home Controller System must satisfy. These requirements govern system behavior across all operational conditions and are essential for formal validation and security verification.

NFR ID	Category	Requirement Statement	Priority
NFR-01	Security	The system shall enforce user authentication using hashed passwords. All unauthorized access attempts shall be logged. Device commands shall be accepted only from authenticated sessions.	Critical
NFR-02	Reliability	The system shall operate without unexpected state transitions or crashes during normal operation. System invariants must hold at all times.	High
NFR-03	Availability	The system shall be available for operation 99% of the time during simulation. Recovery from fault states shall complete within 5 seconds.	High

NFR-04	Performance	The system shall process any user command (e.g., lock door, toggle light) and return a response within 2 seconds under normal conditions.	High
NFR-05	Scalability	The system architecture shall support expansion to at least 10 additional smart device modules without requiring redesign of the core state model.	Medium
NFR-06	Maintainability	The system specification shall be formally structured such that any module can be independently updated or replaced. All operations shall be documented with preconditions and postconditions.	Medium
NFR-07	Usability	System operations and state changes shall be clearly communicated to users via descriptive messages. Users shall be able to complete primary tasks without external guidance.	Medium
NFR-08	Consistency	The system shall never allow simultaneous conflicting states for any device. State invariants shall be preserved across all operations and state transitions.	Critical
NFR-09	Traceability	Every functional requirement shall be traceable to at least one system operation, use case, and formal invariant defined in this document.	High
NFR-10	Verifiability	All system requirements shall be formally verifiable using Z Notation, VDM specification, or Alloy modeling. Ambiguous or incomplete requirements are prohibited.	High

7. System States and State Transitions

7.1 Defined Device States

The IoT Smart Home Controller System defines a finite set of valid states for each managed device. Each device is always in exactly one valid state. The following subsections formally enumerate all permissible states.

7.1.1 Light States

- ON: The smart light is currently illuminated. This state is reached by executing TurnOnLight() when the precondition lightState = OFF is satisfied.
- OFF: The smart light is currently extinguished. This is the default initial state. Reached by executing TurnOffLight() when lightState = ON.

7.1.2 Door States

- LOCKED: The smart door lock is engaged. Physical access through the door is prevented. This is the default initial state for security. Reached via LockDoor().
- UNLOCKED: The smart door lock is disengaged. Physical access is permitted. Reached via UnlockDoor().

7.1.3 Alarm States

- ACTIVE: The alarm is sounding. An alert has been generated. This state may be triggered manually via ActivateAlarm() or automatically upon intrusion detection.
- INACTIVE: The alarm is silent. No alert is active. This is the default initial state. Reached via DeactivateAlarm().

7.1.4 Temperature States

- SAFE: The current temperature reading is at or below the defined safety threshold (default: 40°C). Normal operation continues.
- HIGH: The current temperature reading exceeds the safety threshold. An alert is generated and the temperature state is set to HIGH.

7.2 State Transition Table

The following table describes all defined state transitions, the triggering events for each transition, and the resulting system behavior. These transitions map directly to Z Notation operation schemas and VDM pre/postcondition specifications.

Module	Current State	Next State	Triggering Event / Condition	Resulting Behavior
Light	ON	OFF	User issues TurnOnLight() command and is authenticated.	Light is illuminated; system records light state as ON.
Light	OFF	ON	User issues TurnOffLight() command and is authenticated.	Light is extinguished; system records light

				state as OFF.
Door Lock	UNLOCKED	LOCKED	User issues LockDoor() command and is authenticated.	Door is secured; state changes to LOCKED.
Door Lock	LOCKED	UNLOCKED	User issues UnlockDoor() command and is authenticated.	Door is released; state changes to UNLOCKED.
Alarm	INACTIVE	ACTIVE	ActivateAlarm() is called or intrusion is detected by the system.	Alarm sounds; state changes to ACTIVE; alert is generated.
Alarm	ACTIVE	INACTIVE	DeactivateAlarm() is called by an authenticated user or admin.	Alarm silenced; state changes to INACTIVE.
Temperature	SAFE	HIGH	Sensor reading exceeds predefined threshold (e.g., >40°C).	Temperature state changes to HIGH; alert notification generated.
Temperature	HIGH	SAFE	Sensor reading drops back to or below threshold value.	Temperature state reverts to SAFE; alert cleared.

8. Business Rules and Constraints

The following business rules and constraints define the logical boundaries of the IoT Smart Home Controller System. They serve as the foundation for formal invariants, Alloy assertions, and Z state schema constraints in subsequent verification phases.

9. Mutual Exclusivity of Door States: The smart door lock shall always reside in exactly one of two valid states—LOCKED or UNLOCKED. No operation shall result in the door being simultaneously or neither in both states. This constraint is formally expressed as doorState = LOCKED XOR doorState = UNLOCKED.

10. Alarm Activation on Intrusion Detection: Whenever the system detects an unauthorized access attempt on the door lock, the alarm state shall unconditionally transition to ACTIVE. This constraint cannot be overridden by a standard user command during an active intrusion event.
11. Authentication Prerequisite for All Device Operations: No device control operation (TurnOnLight, LockDoor, ActivateAlarm, etc.) shall be executed by any user who has not successfully completed the authentication process. The authentication state must be TRUE as a mandatory precondition for every operation except Login() and system-level intrusion detection.
12. Temperature Alert Activation Threshold: The temperature monitoring module shall generate an alert and transition the temperature state to HIGH if and only if the sensor reading strictly exceeds the predefined threshold value (default: 40°C). Readings at or below the threshold shall not trigger an alert.
13. Device State Validity Invariant: At no point in time shall any device exist in an undefined, null, or logically impossible state. Each device must always be in one of its enumerated valid states as defined in Section 7.
14. Exclusive Alarm State: The alarm state is always exactly one of ACTIVE or INACTIVE. The system shall reject any operation or event that would result in a null or ambiguous alarm state.
15. Light State Exclusivity: The smart light shall always be in exactly one of ON or OFF states. Transitioning from ON to ON or OFF to OFF is a no-operation and shall return an appropriate status message rather than an error.
16. Admin Authority for Reset Operations: The ResetDevice() operation shall be restricted exclusively to administrators. Standard users shall be denied access to this operation, and such denial shall be logged.
17. Log Immutability: Entries in the audit log are append-only. No user or administrator may modify or delete existing log entries.
18. Valid Sensor Data Requirement: The system shall only update the temperature state based on sensor readings that fall within a physically valid range (e.g., -50°C to 150°C). Readings outside this range shall be discarded and flagged as sensor errors without updating system state.

9. System Invariants

System invariants are logical properties that must hold true across all reachable states of the IoT Smart Home Controller System, regardless of the sequence of operations executed. These invariants are critical for formal verification and are directly expressible as Z schema invariants, VDM state constraints, and Alloy facts or assertions.

ID	Invariant Name	Description	Formal Expression
INV-01	Mutual Exclusivity of Door States	At any point in time, the door state must be exactly one of LOCKED or UNLOCKED. It is logically impossible for	doorState = LOCKED ⊕ doorState = UNLOCKED

		both states to hold concurrently.	
INV-02	Mutual Exclusivity of Alarm States	The alarm must always reside in exactly one of two states: ACTIVE or INACTIVE. No third or null state is permissible.	alarmState = ACTIVE ⊕ alarmState = INACTIVE
INV-03	Mutual Exclusivity of Light States	The light must always be in exactly one of its two valid states: ON or OFF. Intermediate states are not permitted.	lightState = ON ⊕ lightState = OFF
INV-04	Authentication Prerequisite for Device Control	No device state transition may be initiated unless the requesting user's authentication status is TRUE.	∀ op ∈ Operations: execute(op) ⇒ isAuthenticated(user)
INV-05	Intrusion Detection Triggers Alarm	Whenever an intrusion event is detected by the system, the alarm state must transition to or remain in ACTIVE state.	intrusionDetected ⇒ alarmState = ACTIVE
INV-06	Temperature State Consistency	The temperature state must always reflect the most recent sensor reading: HIGH if the reading exceeds the threshold, SAFE otherwise.	tempReading > threshold ⟺ tempState = HIGH
INV-07	Non-Nullity of Device States	No device may exist in an undefined or null state at any time. Each device is always in one of its predefined valid states.	∀ d ∈ Devices: d.state ≠ null ∧ d.state ∈ ValidStates(d)

9.1 Invariant Explanations

INV-01: Mutual Exclusivity of Door States

A door lock must always be in a definitive state. In any real-world security system, a lock is either engaged or not. Allowing any ambiguity in this state would render the Door Lock Module meaningless from a security perspective. This invariant directly maps to a Z state schema constraint and an Alloy fact.

INV-02: Mutual Exclusivity of Alarm States

The alarm is a binary security device. It is either alerting or silent. An alarm in an undefined state provides no security guarantee. This invariant ensures that the alarm is always in a known, predictable condition.

INV-03: Mutual Exclusivity of Light States

While lights may appear less safety-critical, maintaining state exclusivity ensures the formal model remains consistent. Undefined light states could propagate inconsistencies into the broader state model during verification.

INV-04: Authentication Prerequisite

This invariant is the cornerstone of the system's access control model. It guarantees that no unauthorized entity can alter the physical state of any device. Every Z operation schema for device control will include this as a precondition.

INV-05: Intrusion Triggers Alarm

This invariant captures the fundamental safety rule of the security subsystem. It ensures that the system always responds correctly to a detected intrusion, regardless of the current user interaction state.

INV-06: Temperature State Consistency

The temperature state must faithfully reflect the physical environment. This invariant prevents scenarios where the system reports SAFE while the temperature is in fact above the threshold, which would represent a critical monitoring failure.

INV-07: Non-Nullity of Device States

This meta-invariant asserts that the formal state model is always well-defined. It is essential for Alloy model soundness and Z schema completeness. Any state that becomes undefined would invalidate all invariants that reference it.

10. System Operations

This section defines all system operations with their formal pre- and postconditions. These specifications are designed to map directly into Z Notation operation schemas and VDM function definitions. Each operation is atomic: it either completes successfully and satisfies its postcondition, or it fails and leaves the system state unchanged.

Operation	Purpose	Preconditions	Postconditions	Expected Behavior
Login()	Authenticate a user attempting to access the system.	User provides valid username and password. User is not already authenticated.	Session is created. User gains access to device controls. Authentication state is set to TRUE.	Returns session token on success; returns error message on failure.
LockDoor()	Transition the door	User is authenticated.	Door state changes to	Confirmation message

	state from UNLOCKED to LOCKED.	Door is currently in UNLOCKED state.	LOCKED. System records the transition with timestamp.	returned. Door invariants remain satisfied.
UnlockDoor()	Transition the door state from LOCKED to UNLOCKED.	User is authenticated. Door is currently in LOCKED state.	Door state changes to UNLOCKED. System records the transition.	Confirmation message returned. Door invariants remain satisfied.
TurnOnLight()	Transition the light state from OFF to ON.	User is authenticated. Light is currently in OFF state.	Light state changes to ON. System records the new state.	Confirmation message returned. No conflicting light state exists.
TurnOffLight()	Transition the light state from ON to OFF.	User is authenticated. Light is currently in ON state.	Light state changes to OFF. System records the new state.	Confirmation message returned.
ActivateAlarm()	Transition the alarm from INACTIVE to ACTIVE.	User or admin is authenticated, or intrusion event has been detected.	Alarm state changes to ACTIVE. Alert notification is generated.	Alarm triggers; alert message sent to admin and user.
DeactivateAlarm()	Transition the alarm from ACTIVE to INACTIVE.	User or admin is authenticated. Alarm is currently ACTIVE. No active intrusion event is ongoing.	Alarm state changes to INACTIVE. Alert is cleared.	Confirmation message returned. Alarm stops.
MonitorTemperature()	Read current temperature and update system state accordingly.	Temperature sensor is operational and returning valid readings.	If temperature > threshold: temperature state set to HIGH and alert generated. Else: state remains or reverts to SAFE.	Temperature state updated. Alert generated if threshold exceeded.
Logout()	Terminate the current authenticated session.	User is currently authenticated.	Session is invalidated. Authentication state is set to FALSE.	User returned to unauthenticated state; session token revoked.

ResetDevice()	Reset a specified device to its default safe state.	Admin is authenticated. Device ID is valid. Device is in a recoverable state.	Device state reverts to its predefined default (e.g., Light=OFF, Door=LOCKED, Alarm=INACTIVE).	Confirmation message; device state log updated.

11. Use Cases

This section presents detailed use case specifications for the primary user interactions with the IoT Smart Home Controller System. Each use case is structured with a unique identifier, actor specification, preconditions, main flow, alternate flow, and postconditions.

Use Case 1: User Login

Use Case ID	UC-01
Use Case Name	User Login
Actor(s)	User
Preconditions	The user has access to the system interface. The user has a valid registered account with username and password.
Main Flow	1. User navigates to the login interface. 2. User enters username and password. 3. System validates credentials against stored records. 4. On match, system creates a session and issues a session token. 5. User is granted access to device control operations.
Alternate Flow	3a. Credentials are invalid: System displays 'Authentication Failed' message. Attempt is logged. If 5 consecutive failures occur, account is temporarily locked for 15 minutes. 3b. Account is locked: System displays lockout message and denies login.
Postconditions	User is authenticated. A valid session token exists. User can now execute device control operations. Authentication state = TRUE.

Use Case 2: Turn ON Light

Use Case ID	UC-02
Use Case Name	Turn ON Smart Light

Actor(s)	User
Preconditions	User is authenticated (session token is valid). Smart light is currently in OFF state.
Main Flow	1. Authenticated user issues TurnOnLight() command. 2. System verifies user authentication status. 3. System checks that lightState = OFF. 4. System transitions lightState to ON. 5. System records state change with timestamp. 6. System returns confirmation message to user.
Alternate Flow	2a. User is not authenticated: System rejects command with 'Access Denied' message and logs the attempt. 3a. Light is already ON: System returns informational message 'Light is already ON' without altering state.
Postconditions	lightState = ON. State change is recorded. User receives confirmation.

Use Case 3: Lock Door

Use Case ID	UC-03
Use Case Name	Lock Smart Door
Actor(s)	User
Preconditions	User is authenticated. Door is currently in UNLOCKED state.
Main Flow	1. Authenticated user issues LockDoor() command. 2. System verifies authentication. 3. System verifies doorState = UNLOCKED. 4. System transitions doorState to LOCKED. 5. State change is recorded with timestamp. 6. Confirmation returned to user.
Alternate Flow	2a. User is unauthenticated: Command rejected; attempt logged; intrusion detection triggered. 3a. Door is already LOCKED: System returns 'Door is already LOCKED' message without state change.
Postconditions	doorState = LOCKED. System invariant INV-01 is satisfied. State change recorded.

Use Case 4: Activate Alarm

Use Case ID	UC-04
Use Case Name	Activate Smart Alarm
Actor(s)	User / Admin / System (on intrusion)

Preconditions	Actor is authenticated (for manual activation), OR an intrusion event has been detected by the system (for automatic activation). Alarm is currently INACTIVE.
Main Flow	1. Actor issues ActivateAlarm() or intrusion event triggers activation. 2. System checks preconditions. 3. System transitions alarmState to ACTIVE. 4. System generates alert notification to user and admin. 5. State change recorded. 6. Confirmation returned (if manual).
Alternate Flow	Alarm already ACTIVE: System returns 'Alarm is already ACTIVE' without state change.
Postconditions	alarmState = ACTIVE. Alert generated. State change recorded. INV-02 satisfied.

Use Case 5: Monitor Temperature

Use Case ID	UC-05
Use Case Name	Monitor Temperature
Actor(s)	System (automatic)
Preconditions	Temperature monitoring module is operational. Sensor is returning valid readings within physically valid range.
Main Flow	1. System continuously reads temperature sensor at regular intervals. 2. System validates sensor reading (range check). 3a. If reading > 40°C: System transitions tempState to HIGH; generates temperature alert notification. 3b. If reading <= 40°C: System ensures tempState = SAFE; no alert generated. 4. State change (if any) is recorded with timestamp.
Alternate Flow	Sensor returns invalid reading: Reading discarded; sensor error logged; tempState not modified. System returns to step 1.
Postconditions	tempState reflects current temperature condition (SAFE or HIGH). Alert generated if HIGH. INV-06 satisfied.

12. State Transition Descriptions

This section provides narrative descriptions of all state transitions defined in the system. These descriptions supplement the State Transition Table in Section 7.2 and provide the contextual detail required for formal modeling in Z Notation and Alloy.

12.1 Light State Transitions

OFF → ON (TurnOnLight):

Triggered when an authenticated user issues the TurnOnLight() command while lightState = OFF. The system verifies authentication, confirms the current state, and atomically transitions lightState to ON. The transition is recorded in the state log. If lightState is already ON, the command is treated as a no-operation.

ON → OFF (TurnOffLight):

Triggered when an authenticated user issues the TurnOffLight() command while lightState = ON. The system verifies authentication, confirms the current state, and atomically transitions lightState to OFF. Recorded in the state log.

12.2 Door Lock State Transitions

UNLOCKED → LOCKED (LockDoor):

Triggered by an authenticated user issuing LockDoor() when doorState = UNLOCKED. The system verifies all preconditions and atomically transitions doorState to LOCKED. Upon completion, INV-01 is verified to remain satisfied. If the command originates from an unauthenticated session, the request is rejected and an intrusion event is raised.

LOCKED → UNLOCKED (UnlockDoor):

Triggered by an authenticated user issuing UnlockDoor() when doorState = LOCKED. Preconditions verified, state transitioned atomically. INV-01 verified post-transition.

12.3 Alarm State Transitions

INACTIVE → ACTIVE (ActivateAlarm or Intrusion Event):

Triggered either manually by an authenticated user or admin, or automatically by the system upon detection of an intrusion event. This is the only transition that may be initiated without direct user command. Once transitioned, alarmState = ACTIVE and an alert is dispatched.

ACTIVE → INACTIVE (DeactivateAlarm):

Triggered exclusively by an authenticated user or admin issuing DeactivateAlarm() when alarmState = ACTIVE and no ongoing intrusion event is active. The alarm is silenced and alarmState is set to INACTIVE.

12.4 Temperature State Transitions

SAFE → HIGH:

Occurs automatically when the temperature monitoring module reads a sensor value strictly greater than the threshold (40°C). The system transitions tempState to HIGH and generates a temperature alert notification. This transition does not require user input.

HIGH → SAFE:

Occurs automatically when the sensor reading drops to or below the threshold value. The tempState is reverted to SAFE and any active temperature alert is cleared.

13. Requirement Defect Taxonomy

This section identifies defects discovered during the requirements engineering phase of the IoT Smart Home Controller System. The defect taxonomy classifies each identified issue by type, describes its impact on formal verification, and provides the corrective action taken. This analysis is an essential component of the Software Verification and Validation methodology.

Defect ID	Type	Req. ID / Description	Impact	Suggested Correction
DT-01	Ambiguity	FR-05 originally stated 'high temperature' without specifying a numeric threshold, making it unmeasurable and unverifiable.	Cannot be formally modeled; formal invariant cannot be defined.	Requirement revised to specify a concrete threshold: temperature > 40°C triggers HIGH state.
DT-02	Inconsistency	FR-07 prohibited unauthorized access, but FR-08 implied the alarm is triggered without an explicit authenticated user command, creating a logical contradiction.	Formal operation ActivateAlarm() would have conflicting preconditions.	FR-08 revised to explicitly allow system-triggered alarm activation as a separate autonomous operation.
DT-03	Non-Verifiability	Early draft of FR-03 stated the door 'should be secure,' a qualitative statement that cannot be formally verified.	No formal postcondition can be derived; invariant cannot be specified.	Revised to: UnlockDoor() transitions door state from LOCKED to UNLOCKED, verifiable via state schema.

DT-04	Missing Constraint	Initial requirements did not specify that a door cannot exist in both LOCKED and UNLOCKED states simultaneously.	State model would permit invalid concurrent states; Alloy model would be unsound.	System invariant INV-01 added: doorState = LOCKED XOR doorState = UNLOCKED at all times.
DT-05	Incomplete Requirement	FR-04 described alarm activation but did not define the condition for automatic deactivation, leaving postconditions incomplete.	Z operation schema for DeactivateAlarm() would be underspecified.	FR-04 updated to include both activation and deactivation logic with full preconditions and postconditions.
DT-06	Ambiguity	FR-10 stated admin can 'manage users' without defining what management operations are permitted (create, delete, modify).	Scope creep risk; formal modeling of admin operations is impossible.	FR-10 scoped to: view device logs and view user account list. Write operations deferred to future work.
DT-07	Missing Constraint	No original requirement specified what happens if a temperature sensor returns an out-of-range or null value.	System could enter undefined state; formal model would be incomplete.	NFR-08 updated and FR-05 revised to specify that invalid sensor readings are rejected and system state is not updated.

14. Validation Requirements

Validation requirements define the specific criteria that must be confirmed to demonstrate that the IoT Smart Home Controller System meets its stated requirements. Each validation criterion is traceable to one or more functional requirements and system invariants. This checklist will form the basis for the validation phase of the SVV project.

ID	Traced Req.	Validation Criterion	Verification Method	Expected Result
VC-01	FR-03, INV-01	Verify that LockDoor() transitions door state from UNLOCKED to LOCKED and that no concurrent conflicting state exists.	State inspection after operation execution	Pass
VC-02	FR-04, INV-02	Verify that ActivateAlarm() transitions alarm from INACTIVE to ACTIVE and generates an alert notification.	Event trace and state inspection	Pass
VC-03	FR-01, INV-04	Verify that unauthenticated users are denied access to all device control operations.	Boundary test with no session token	Pass
VC-04	FR-02, INV-03	Verify that TurnOnLight() and TurnOffLight() correctly toggle the light state and maintain mutual exclusivity.	State inspection before and after operations	Pass
VC-05	FR-08, INV-05	Verify that an intrusion detection event automatically triggers alarm activation without requiring a manual user command.	Simulate intrusion event; inspect alarm state	Pass
VC-06	FR-09, INV-06	Verify that when temperature reading exceeds 40°C, the temperature state transitions to HIGH and an alert is generated.	Inject out-of-threshold sensor value	Pass
VC-07	FR-05, INV-07	Verify that no device state becomes null or undefined under any sequence of valid operations.	Exhaustive state trace analysis	Pass
VC-08	FR-11	Verify that the system rejects any command that would place the	Submit conflicting state commands	Reject

		door in an invalid concurrent state.		
VC-09	FR-10	Verify that admin can retrieve device status logs without altering system state.	Admin log retrieval; state comparison	Pass
VC-10	FR-12	Verify that ResetDevice() restores any device to its default safe state and that the post-reset state satisfies all invariants.	Execute reset; verify default state	Pass

15. Security Requirements

The following security requirements define the protection mechanisms that the IoT Smart Home Controller System must implement. These requirements are aligned with the OWASP (Open Web Application Security Project) Top 10 vulnerability taxonomy and are structured to support security verification using tools such as OWASP ZAP in applicable simulation contexts.

ID	Category	Requirement	OWASP Ref.	Priority
SR-01	Authentication Validation	The system shall validate user credentials against stored hashed values using a secure one-way hashing algorithm. Plain-text passwords shall never be stored or transmitted.	OWASP A07	Critical
SR-02	Unauthorized Access Prevention	The system shall enforce session-based access control. Any request without a valid active session token shall be rejected and logged.	OWASP A01	Critical
SR-03	Input Validation	All user inputs (usernames, passwords, device commands) shall be validated for type, length, and format before processing. Malformed inputs shall be rejected.	OWASP A03	High

SR-04	Session Management	User sessions shall expire automatically after a predefined inactivity period. Session tokens shall be revoked upon logout or timeout.	OWASP A07	High
SR-05	Device Command Authorization	Each device command shall be verified for user authorization level before execution. Admin-only commands shall reject execution by standard users.	OWASP A01	Critical
SR-06	Intrusion Detection Logging	All failed authentication attempts and unauthorized access attempts shall be logged with timestamp, user identifier, and attempted action for audit purposes.	OWASP A09	High
SR-07	State Integrity Enforcement	The system shall not execute any operation that would result in a violation of a defined system invariant. Such operations shall be rejected with an appropriate error.	System Safety	Critical
SR-08	Denial of Service Mitigation	The system shall limit the number of consecutive failed login attempts to five before temporarily locking the account for 15 minutes.	OWASP A07	Medium

16. Assumptions and Limitations

16.1 Assumptions

The following assumptions have been made during the specification of the IoT Smart Home Controller System:

19. The system operates within a logically stable and consistent simulation environment. Network connectivity interruptions and hardware failures are outside the scope of this specification.
20. All sensor inputs (particularly temperature) are assumed to return valid numeric values within a physically plausible range during normal simulation operation.
21. The formal verification tools (Z/EVES, Alloy Analyzer, VDM Tools) used in subsequent project phases are assumed to operate correctly and in accordance with their documented specifications.

22. The system specification assumes a single concurrent user per session for simplicity. Concurrent multi-user access is not modeled in this version.
23. User credentials are assumed to be stored in a secure, pre-populated user record at system initialization. User registration is outside the scope of this specification.
24. The predefined temperature threshold of 40°C is assumed to be appropriate for the simulation context and may be adjusted without altering the fundamental formal model.

16.2 Limitations

The following limitations apply to the current version of the IoT Smart Home Controller System specification:

25. This specification describes a simulation-only system. No real physical hardware, embedded systems, network protocols, or IoT communication standards (e.g., MQTT, Zigbee) are incorporated.
26. The system is limited to four device categories (Light, Door Lock, Alarm, Temperature Monitor). Extension to additional device types requires corresponding updates to the formal state model and operation specifications.
27. Concurrent operations by multiple users are not formally specified. The model assumes sequential, non-overlapping operation execution.
28. The system does not model communication latency, sensor drift, or transient hardware failures. These are considered implementation concerns beyond the scope of formal specification.
29. The security requirements are specified at the architectural level and are not tied to a specific cryptographic implementation. Actual cryptographic protocol selection is deferred to implementation.

17. Future Enhancements

The IoT Smart Home Controller System, as specified in this document, provides a formally verifiable foundation that can be extended in future development cycles. The following enhancements are proposed for consideration in subsequent versions:

30. Mobile Application Integration: Development of a dedicated mobile application (iOS and Android) to provide users with a graphical interface for device control, real-time status monitoring, and alert reception, replacing the current abstract interface specification.
31. AI-Driven Automation: Integration of machine learning models to enable predictive and automated device control based on user behavioral patterns, time-of-day schedules, and environmental conditions.
32. Voice Assistant Integration: Support for voice command interfaces (e.g., Amazon Alexa, Google Assistant) to enable hands-free operation of smart home devices through natural language commands.

33. Cloud Synchronization: Migration of the device state store and audit logs to a cloud-based backend, enabling remote access, cross-device synchronization, persistent storage, and scalable multi-home management.
34. Multi-User Concurrent Access: Extension of the formal model to support concurrent multi-user sessions with conflict resolution, role-based access control, and formal concurrency verification.
35. Physical Hardware Integration: Development of an actual IoT prototype using microcontrollers (e.g., Raspberry Pi, Arduino) and real sensors to validate the formal model against physical system behavior.
36. Energy Management Module: Addition of a new device category for smart energy management, including monitoring of power consumption, automated load balancing, and threshold-based alerts.
37. Enhanced Security Protocols: Implementation of multi-factor authentication (MFA), end-to-end encryption for device communications, and real-time intrusion detection algorithms.

18. Conclusion

This Software Requirements Specification document provides a comprehensive, formally grounded, and technically rigorous specification of the IoT Smart Home Controller System, developed as a semester project for the Software Verification and Validation course. The document defines the complete functional and non-functional requirements, system states and transitions, operational preconditions and postconditions, system invariants, use cases, and security requirements that collectively characterize the intended behavior of the system.

The central contribution of this SRS lies in its alignment with formal verification methodology. Every requirement, constraint, invariant, and operation has been specified with sufficient precision to support direct translation into formal notations including Z Notation, the Vienna Development Method (VDM), and Alloy relational logic. This precision ensures that the system can be subjected to rigorous mathematical analysis, enabling proof of correctness properties that cannot be guaranteed by conventional testing alone.

The system invariants defined in Section 9 establish the foundational safety and security properties of the system—most critically, the mutual exclusivity of device states, the authentication prerequisite for all control operations, and the automatic alarm activation upon intrusion detection. These invariants will serve as the primary verification targets in subsequent project phases.

The defect taxonomy in Section 13 demonstrates that the requirements engineering process itself benefits from systematic analysis: ambiguous, inconsistent, and non-verifiable requirements were identified and corrected before formal modeling commenced, thereby preventing the propagation of specification errors into the verification phase.

In conclusion, the IoT Smart Home Controller System specification presented in this document provides a sound, complete, and formally verifiable foundation for a reliable and secure smart home management system. The disciplined application of Software Verification and Validation methodology at the requirements stage is expected to yield a system specification whose correctness can be demonstrated with mathematical confidence, and whose formal models will accurately reflect the intended operational behavior of the system across all reachable states.